

Desenvolvimento da Unidade Operativa Uniciclo RISC-V32I

Demétrius Mendes Miranda Rocha
232012974

Lucas Silva Nóbrega
180035096

João Gabriel Melo de Lima
241032617

Marcus Vinicius Balbino Cavalcante
160135974

Samuel Carvalho Dos Santos
211010388

Resumo—Este artigo apresenta o desenvolvimento da unidade operativa de um processador Uniciclo baseado na arquitetura RISC-V32I. O projeto foi implementado utilizando a linguagem de descrição de hardware Verilog e o ambiente de desenvolvimento Quartus II. A arquitetura foi estruturada para suportar as instruções fundamentais do conjunto RV32I, integrando componentes essenciais como Unidade Lógica e Aritmética (ULA), banco de registradores, memórias de instruções e dados, e a unidade de controle. O trabalho demonstra o mapeamento de instruções para sinais de controle, o processamento de dados e a sincronização do caminho de dados (datapath) em um único ciclo de clock. Através de simulações utilizando arquivos de inicialização de memória (.mif), valida-se o correto funcionamento das instruções e estabelece-se a base prática para o estudo da organização de processadores.

Palavras-chave—RISC-V32I, Uniciclo, Verilog, Datapath, Quartus II, Arquitetura de Computadores.

Tabela I
INFORMAÇÕES GERAIS DO LABORATÓRIO

Disciplina	Organização e Arquitetura de Computadores
Turma	03
Professor	Flávio Vidal
Semestre	1º/2026
Laboratório	Laboratório 02
Tema	Unidade Operativa Uniciclo RISC-V32I

I. INTRODUÇÃO

A arquitetura de computadores estuda os princípios de projeto, organização e implementação de sistemas computacionais. No contexto dos processadores, a abordagem Uniciclo representa um dos modelos mais didáticos de organização. Neste modelo, cada instrução completa sua execução ao longo de um único ciclo de relógio (clock), de forma que o período do clock deve ser longo o suficiente para acomodar o caminho crítico mais demorado, tipicamente correspondente à instrução de acesso à memória de dados, como o `lw` (Load Word).

O conjunto de instruções RISC-V tem se tornado um padrão proeminente em meios acadêmicos e industriais devido à sua natureza aberta (open-source), extensibilidade e design limpo, baseado nos princípios RISC (Reduced Instruction Set Computer). A vertente RV32I, em particular, especifica o conjunto básico de 32 bits para números inteiros, servindo como base

para projetos educacionais de processadores e implementações embarcadas [1].

Este projeto descreve a implementação completa do caminho de dados (datapath) e da unidade de controle de um processador Uniciclo RISC-V32I em Verilog. O processador foi desenvolvido para simular o comportamento de instruções fundamentais, manipulando dados a partir de memórias implementadas segundo a arquitetura Harvard (memórias separadas para dados e instruções). O ambiente de desenvolvimento Quartus II foi utilizado para a integração e validação dos módulos funcionais.

II. OBJETIVOS

O principal objetivo deste trabalho é proporcionar a familiarização com a construção, o controle e a execução de instruções na Unidade Operativa de um processador Uniciclo RISC-V32I.

De forma específica, busca-se:

- Implementar os módulos principais do *datapath* em Verilog (ULA, Unidade de Controle, Memórias, Geração de Imediatos, entre outros).
- Interligar os módulos respeitando os caminhos de dados e os sinais de controle adequados.
- Processar conjuntos de instruções de 32 bits pré-compiladas (arquivos `.mif` gerados no laboratório anterior).
- Analisar e simular o fluxo de dados durante a execução de instruções de acesso à memória, operações aritméticas e de desvio.
- Avaliar o desempenho estimando tempos de propagação nas diferentes unidades funcionais.

III. FUNDAMENTAÇÃO TEÓRICA

A. Caminho de Dados (Datapath) e Arquitetura Uniciclo

O caminho de dados (*datapath*) é o conjunto de elementos funcionais que processam informações no processador. Ele inclui componentes de armazenamento temporário, como o banco de registradores (RegFile) e o Contador de Programa (PC - Program Counter), além de componentes de transformação e roteamento, como a Unidade Lógica e Aritmética (ULA) e os multiplexadores. Na abordagem uniciclo, todas as etapas (busca, decodificação, execução, acesso à memória

e escrita) são executadas no mesmo ciclo de clock, exigindo que o *datapath* contenha recursos dedicados para cada etapa para evitar conflitos estruturais, como memórias separadas para dados e instruções (Arquitetura Harvard).

B. A Unidade de Controle

A unidade de controle atua orquestrando o fluxo de dados pelos componentes do processador. No RISC-V, ela extrai informações contidas nos campos da instrução, como o *opcode* (bits 6:0), e subcampos como *funct3* (bits 14:12) e *funct7* (bits 31:25) para determinar a operação da ULA, controlar a escrita no banco de registradores (*RegWrite*), habilitar leitura/escrita de memórias (*MemRead*, *MemWrite*), e controlar os multiplexadores (*ALUSrc*, *MemtoReg*, *Branch*, etc.). O processador inclui tanto um Controle Principal, baseado primariamente no *opcode*, quanto um Controle da ULA (*ALU Control*), dependente do *funct3*, *funct7* e de sinais do Controle Principal.

IV. MATERIAIS E MÉTODOS

A. Ferramentas Utilizadas

O desenvolvimento foi realizado utilizando a linguagem de descrição de hardware Verilog (IEEE 1364). A integração, síntese e simulação dos módulos foram realizadas por meio do *software* Quartus II. Para a simulação, fez-se uso intensivo do ambiente Waveform Editor (Vector Waveform) e da opção de simulação *Timing*, permitindo não só validar os sinais lógicos como também avaliar atrasos de propagação no circuito.

B. Módulos de Memória e Integração

Seguindo os requisitos propostos, o processador utiliza uma estrutura Harvard com dois blocos de memória distintos:

- **Memória de Instruções:** Carregada com o arquivo *UnicicloInst.hex* ou *.mif*, contendo os mnemônicos do programa montado pelo assembler.
- **Memória de Dados:** Carregada com o arquivo *UnicicloData.hex* ou *.mif*, responsável por manter as variáveis e estruturas estáticas durante a execução do programa.

O código foi estruturado em módulos independentes (*alu.v*, *control.v*, *reg_file.v*, etc.), que foram posteriormente instanciados e conectados em um módulo topo denominado *riscv_top.v*. Esta arquitetura modular facilita testes unitários de cada componente (ex: testar as funções de soma da ULA independentemente) antes da integração no caminho de dados completo.

V. IMPLEMENTAÇÃO DO PROCESSADOR

A implementação foi dividida no desenvolvimento e conexão dos blocos funcionais do Datapath. Abaixo são descritos os componentes desenvolvidos:

A. Banco de Registradores (*reg_file.v*)

O arquivo de registradores modela o conjunto de 32 registradores inteiros (*x0* a *x31*). O módulo possui duas portas de leitura assíncronas e uma porta de escrita síncrona. O registrador *x0* foi fixado logicamente em zero, descartando-se qualquer tentativa de escrita no seu endereço.

B. Unidade Lógica Aritmética e Geração de Imediatos

A ULA (*alu.v*) foi projetada para receber um sinal de controle da operação a ser realizada (*ALUControl*) e executar instruções de soma, subtração, operações lógicas (AND, OR, XOR) e deslocamentos lógicos e aritméticos. O módulo emissor do sinal de zero (*Zero flag*) foi acoplado à saída para subsidiar o controle de ramificações condicionais (*branch*).

A unidade de geração de immediatos (*imm_gen.v*) recebe a instrução de 32 bits completa e reconstrói o valor imediato em 32 bits sinalizados, ajustando a extração dos bits conforme o formato da instrução (I, S, B, U ou J).

C. Unidade de Controle (*control.v* e *alu_control.v*)

O bloco *control.v* foi desenvolvido baseado em uma máquina combinacional que recebe os 7 bits de *opcode* e aciona os sinais responsáveis pelo fluxo de informações, como *Branch*, *MemRead*, *MemtoReg*, *MemWrite*, *ALUSrc*, *RegWrite* e *ALUOp*. O *alu_control.v* utiliza os sinais de *ALUOp* recebidos em conjunto com os bits *funct3* e o bit 30 (*funct7[5]*) para definir a instrução precisa da ULA (ex: diferenciar uma soma *add* de uma subtração *sub*, que compartilham o mesmo *opcode* e *funct3*).

D. Módulo Topo (*riscv_top.v*)

Este é o módulo que instanciou e interligou todos os submódulos, juntamente com a lógica de *Program Counter* e os multiplexadores. O PC (Contador de Programa) foi implementado como um registrador síncrono que é atualizado a cada ciclo. Lógicas combinacionais adicionais foram implementadas para calcular os endereços de desvio (soma do PC atual com o imediato deslocado esquematicamente). Este módulo suporta a execução fluída das seguintes classes de instruções solicitadas: Acesso à memória (*lw*, *sw*, *lhu*), aritmético-lógicas entre registradores (*add*, *sub*, *and*, *or*, *xor*, *sll*, *srl*, *slt*), aritmético-lógicas com immediatos (*addi*, *andi*, *ori*, *xori*, *slti*), desvios condicionais (*beq*, *bne*), endereçamento superior (*lui*, *auipc*) e saltos incondicionais (*jal*, *jalr*).

VI. RESULTADOS E DISCUSSÃO

A. Simulação e Execução

A arquitetura foi validada através do processamento da carga dos arquivos do laboratório anterior. As simulações confirmaram que as instruções buscam dados nos registradores corretos, geram immediatos de acordo com as extensões de sinal adequadas, operam na ULA sem produzir falsos resultados, acessam a memória nos ciclos devidos e, caso possuam *RegWrite* verdadeiro, confirmam o registro final no banco de registradores antes da subida do próximo *clock*.

VII. CONCLUSÃO

A execução prática do Laboratório 2 logrou êxito na integração e funcionamento da unidade operativa do Uniciclo RISC-V32I. Foi possível compreender de maneira profunda o papel da unidade de controle na modulação do trajeto da informação. Ao implementar e simular cada etapa, o estudo demonstrou como uma representação textual e abstrata de um programa é desfragmentada pelo hardware em sinais elétricos organizados. O Quartus II provou-se uma ferramenta adequada para validar os arranjos de multiplexadores, ULA, registradores e memória. Fica clara a validade deste modelo para fins didáticos, ainda que na prática modernas técnicas de *pipelining* sejam preferidas a fim de evitar o desperdício de tempo inerente ao longo ciclo estático do *datapath* monociclo.

REFERÊNCIAS

- [1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. [Online]. Available: <https://riscv.org/technical/specifications/>
- [2] D. A. Patterson e J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. 1a ed., Morgan Kaufmann, 2017.
- [3] Intel, *Quartus Prime Software Documentation*. [Online]. Available: <https://www.intel.com/content/www/us/en/programmable/documentation>

Figura 1. Resultados da simulação no ambiente Waveform Editor.

Na Figura 1, pode-se observar o comportamento do circuito. Nota-se o *update* correto do registrador PC, bem como o preenchimento de sinais de *MemRead* correspondentes durante instruções de leitura *lw*.

B. Análise de Tempos de Propagação (Gargalos)

Para a identificação dos caminhos críticos, utiliza-se a modalidade de simulação *Timing*. O caminho de dados do tipo uniciclo é limitado pela instrução mais demorada. Para a arquitetura RISC-V, o pior cenário ocorre na instrução *lw* (Load Word), visto que o sinal precisa fluir sequencialmente pelas seguintes etapas: Busca de Instrução (Memória) → Banco de Registradores (leitura do endereço base) → ULA (cálculo de deslocamento do endereço) → Memória de Dados (leitura do dado) → Banco de Registradores (escrita do resultado).

A Tabela II ilustra uma estimativa teórica do tempo propagado por cada unidade funcional, determinando o tempo do ciclo crítico.

Tabela II
ANÁLISE DE TEMPO DE PROPAGAÇÃO POR UNIDADE FUNCIONAL

Unidade Funcional	Tempo Absoluto (ps)	Percentual (%)
Memória de Instruções	250	27.8%
Banco de Registradores (Leitura)	150	16.7%
Unidade Lógica Aritmética (ULA)	200	22.2%
Memória de Dados	250	27.8%
Banco de Registradores (Escrita)	50	5.5%
Tempo Total (Período Crítico)	900	100%

Estes resultados demonstram as limitações da arquitetura de ciclo único no que diz respeito ao atraso de propagação do sinal; instruções inerentemente mais rápidas (como simples saltos incondicionais) são forçadas a aguardar a janela de ciclo imposta pela lentidão estrutural de leitura de memória presente apenas na instrução *lw*. Isso justifica implementações mais avançadas como Pipeline, onde o *clock* não necessita acomodar toda a soma de latências do processador em uma única janela estática.